

INFORME — SISTEMA DE GESTIÓN DE FRANXX

UML, PROGRAMACIÓN ORIENTADA A OBJETOS Y PATRONES DE DISEÑO

Tema: Darling in the FranXX

1. INTRODUCCIÓN

El presente informe desarrolla el diseño orientado a objetos de un sistema destinado a gestionar los FRANXX, sus pilotos, escuadrones y misiones dentro del universo de *Darling in the FranXX*.

Para realizar el diseño se aplican conceptos fundamentales de Programación Orientada a Objetos (POO), UML y patrones de diseño.

Los principales objetivos del sistema son:

- Administrar los diferentes modelos de FRANXX.
- Administrar los pilotos y escuadrones.
- Controlar los estados de los FRANXX.
- Permitir la utilización de diferentes estrategias de combate.
- Registrar las misiones realizadas.
- Notificar a los diferentes sistemas cuando ocurre un cambio importante.
- Permitir agregar nuevos modelos de FRANXX sin modificar considerablemente el código existente.

Para resolver estos requerimientos se utilizan los patrones:

- **Factory**
- **Strategy**
- **State**
- **Observer**

Además, se aplican los conceptos de encapsulamiento, abstracción, herencia y polimorfismo, junto con principios SOLID.

2. IDENTIFICACIÓN DE CLASES

A partir del análisis del problema se identifican las siguientes clases principales:

1. `Franxx`
2. `Strelizia`
3. `Delphinium`
4. `Argentea`

5. Genista
 6. Chlorophytum
 7. Piloto
 8. Escuadron
 9. Mision
 10. Arma
 11. Klaxosaurio
 12. FranxxFactory
 13. EstadoFranxx
 14. EstadoDisponible
 15. EstadoEnMision
 16. EstadoDañado
 17. EstadoReparandose
 18. EstrategiaCombate
 19. EstrategiaAtaque
 20. EstrategiaDefensa
 21. EstrategiaPatrulla
 22. EstrategiaRetirada
 23. ObservadorFranxx
 24. CentroComando
 25. Mantenimiento
 26. RegistroMisiones
-

3. CLASE FRANXX

La clase `Franxx` representa la abstracción general de cualquier FRANXX existente en el sistema.

Se define como una clase abstracta porque no se desea crear directamente un objeto genérico `Franxx`, sino objetos pertenecientes a modelos concretos.

Atributos

```
- id : int
- nombre : String
- nivelEnergia : double
- armaPrincipal : Arma
- pilotos : List<Piloto>
- estado : EstadoFranxx
- estrategia : EstrategiaCombate
- observadores : List<ObservadorFranxx>
```

Métodos

```
+ iniciarMision() : void
+ atacar(Klaxosaurio) : void
+ defender() : void
```

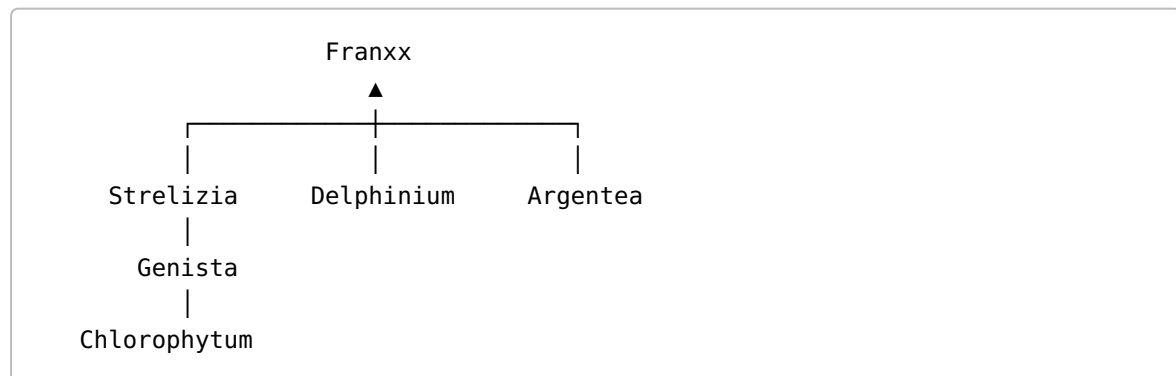
```
+ patrullar() : void
+ retirarse() : void
+ cambiarEstado(EstadoFranxx) : void
+ cambiarEstrategia(EstrategiaCombate) : void
+ ejecutarEstrategia() : void
+ agregarPiloto(Piloto) : void
+ agregarObservador(ObservadorFranxx) : void
+ notificar() : void
```

La clase concentra los comportamientos comunes de todos los FRANXX.

4. HERENCIA DE LOS FRANXX

Los modelos concretos heredan de la clase abstracta `Franxx`.

La estructura propuesta es:



Conceptualmente, cada modelo concreto puede sobrescribir determinados métodos para implementar comportamientos particulares.

Por ejemplo:

```
Franxx
├── Strelizia
├── Delphinium
├── Argentea
├── Genista
└── Chlorophytum
```

Esto permite aplicar **herencia y polimorfismo**.

5. CLASE PILOTO

La clase `Piloto` representa a una persona capaz de pilotar un FRANXX.

Atributos

```
- nombre : String
- codigo : String
- edad : int
- nivelSincronizacion : double
- estadoFisico : String
- compatible : List<Franxx>
```

Métodos

```
+ pilotear(Franxx) : void
+ aumentarSincronizacion() : void
+ verificarCompatibilidad(Franxx) : boolean
+ actualizarEstadoFisico(String) : void
```

Los atributos se mantienen privados para cumplir con el principio de encapsulamiento.

6. CLASE ESCUADRON

La clase `Escuadron` representa un grupo de pilotos que trabaja conjuntamente.

Atributos

```
- codigo : String
- nombre : String
- pilotos : List<Piloto>
- franxx : List<Franxx>
```

Métodos

```
+ agregarPiloto(Piloto) : void
+ asignarFranxx(Franxx) : void
+ iniciarMision(Mision) : void
+ recibirOrden(String) : void
```

Un escuadrón puede tener varios pilotos y varios FRANXX asignados.

7. CLASE MISION

La clase `Mision` representa una misión realizada por un escuadrón.

Atributos

```
- id : int
- tipo : String
- objetivo : String
- fecha : Date
- escuadron : Escuadron
- franxx : Franxx
- estado : String
```

Métodos

```
+ iniciar() : void
+ finalizar() : void
+ cancelar() : void
+ registrarResultado(String) : void
```

8. CLASE ARMA

La clase `Arma` representa el arma utilizada por un FRANXX.

Atributos

```
- nombre : String
- tipo : String
- daño : double
- nivelEnergia : double
```

Métodos

```
+ atacar(Klaxosaurio) : void
+ recargar() : void
+ puedeUtilizarse() : boolean
```

Existe una relación de composición entre `Franxx` y `Arma`, ya que el arma principal pertenece al FRANXX dentro del modelo planteado.

9. CLASE KLAXOSAURIO

Representa al enemigo enfrentado durante las misiones.

Atributos

```
- id : int  
- tipo : String  
- nivelPeligro : int  
- energia : double
```

Métodos

```
+ recibirAtaque(double) : void  
+ atacar() : void  
+ estaDerrotado() : boolean
```

10. RELACIONES ENTRE LAS CLASES

Las principales relaciones son las siguientes:

Escuadron — Piloto

Un escuadrón está compuesto por varios pilotos.

```
Escuadron 1 ----- 2..* Piloto
```

Se utiliza composición porque los pilotos pertenecen al escuadrón dentro del contexto del sistema.

Escuadron — Franxx

Un escuadrón puede administrar varios FRANXX.

```
Escuadron 1 ----- 0..* Franxx
```

Piloto — Franxx

Un piloto puede ser compatible con varios FRANXX y un FRANXX puede tener varios pilotos compatibles.

```
Piloto 0..* ----- 0..* Franxx
```

Franxx — Arma

Cada FRANXX posee un arma principal.

```
Franxx 1 ♦----- 1 Arma
```

Se utiliza composición porque el arma forma parte del FRANXX en el modelo propuesto.

Escuadron — Mision

Un escuadrón puede participar en muchas misiones.

```
Escuadron 1 ----- 0..* Mision
```

Franxx — Mision

Un FRANXX puede participar en muchas misiones a lo largo de su vida operativa.

```
Franxx 1 ----- 0..* Mision
```

11. PATRÓN FACTORY

Para crear los diferentes modelos de FRANXX se utiliza el patrón **Factory**.

La fábrica será responsable de crear el objeto correspondiente según el modelo solicitado.

Clase

```
FranxxFactory
```

Método

```
+ crearFranxx(tipo : String) : Franxx
```

La utilización sería conceptualmente:

```
FranxxFactory.crearFranxx("STRELIZIA")
```

y devolvería un objeto:

Strelizia

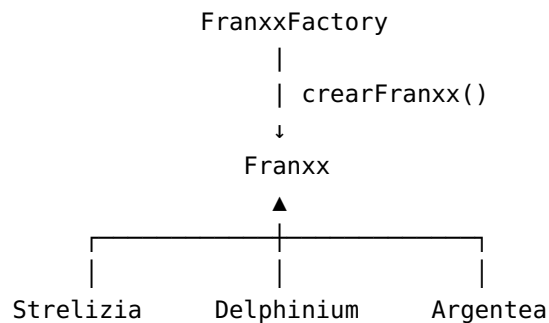
De igual manera:

```
FranxxFactory.crearFranxx("DELPHINIUM")
```

devolvería:

Delphinium

Estructura UML



Justificación

Factory permite separar la lógica de creación de objetos de las clases que utilizan esos objetos.

De esta manera, el resto del sistema no necesita conocer directamente cómo se construye cada modelo de FRANXX.

También facilita agregar nuevos modelos.

12. PATRÓN STRATEGY

El patrón Strategy se utiliza para representar las diferentes estrategias de combate.

Se define una interfaz:

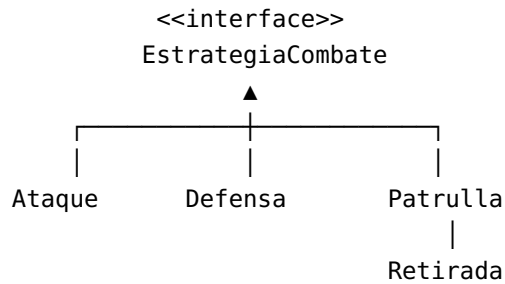
```
<<interface>>
EstrategiaCombate
-----
+ ejecutar(Franxx) : void
```

Las estrategias concretas son:


```
EstrategiaAtaque  
EstrategiaDefensa  
EstrategiaPatrulla  
EstrategiaRetirada
```

Cada una implementa la interfaz `EstrategiaCombate`.

Estructura



El `Franxx` posee una referencia a:

```
EstrategiaCombate
```

Por lo tanto puede cambiar de estrategia durante la ejecución.

Justificación

Strategy permite encapsular diferentes algoritmos de combate y cambiarlos dinámicamente.

Si se utilizara un gran `switch`, la clase `Franxx` tendría que conocer todas las estrategias y crecería cada vez que se agregara una nueva.

Con Strategy, una nueva estrategia puede agregarse creando una nueva clase que implemente la interfaz.

13. PATRÓN STATE

El patrón State se utiliza para representar el estado operativo de los FRANXX.

Se define:

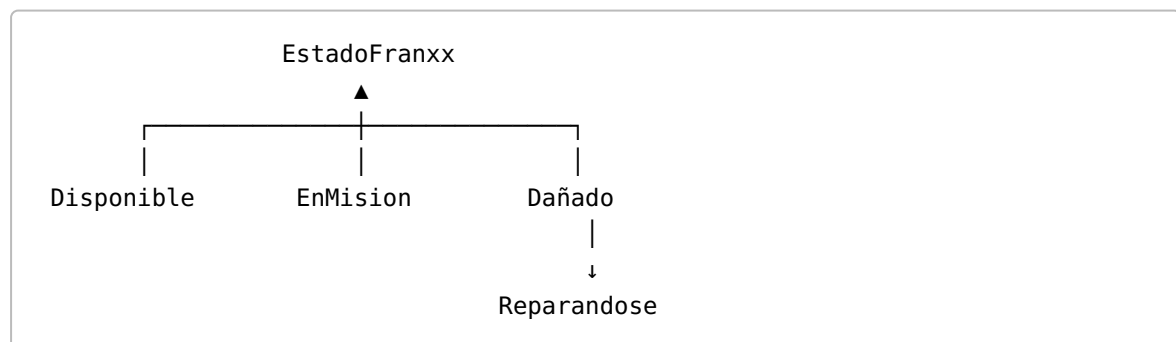
```
<<interface>>  
EstadoFranxx  
-----  
+ iniciarMision(Franxx)
```

```
+ atacar(Franxx)
+ reparar(Franxx)
+ finalizarMision(Franxx)
```

Los estados concretos son:

```
EstadoDisponible
EstadoEnMision
EstadoDañado
EstadoReparandose
```

Diagrama conceptual



Transiciones

```
DISPONIBLE
  ↓ iniciar misión
EN MISIÓN
  ↓ recibir daño
DAÑADO
  ↓ iniciar reparación
REPARÁNDOSE
  ↓ finalizar reparación
DISPONIBLE
```

Justificación

State permite cambiar el comportamiento de un FRANXX dependiendo de su estado actual.

Por ejemplo, un FRANXX dañado no debería poder iniciar una misión.

En lugar de colocar múltiples condiciones dentro de `Franxx`, cada estado encapsula su propio comportamiento.

14. PATRÓN OBSERVER

El patrón Observer se utiliza para notificar a diferentes sistemas cuando un FRANXX cambia de estado.

El FRANXX funciona como **Subject/Observable**.

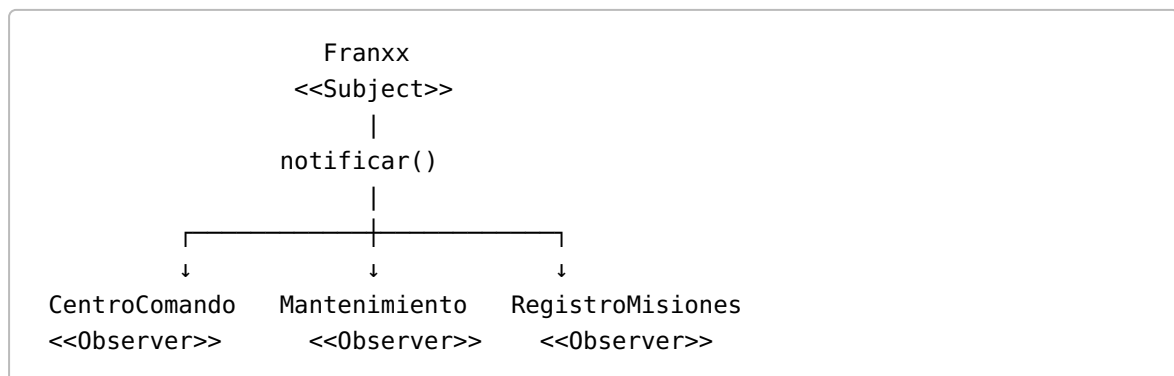
Los observadores son:

```
CentroComando  
Mantenimiento  
RegistroMisiones
```

Se define:

```
<<interface>>  
ObservadorFranxx  
-----  
+ actualizar(Franxx) : void
```

Estructura UML



Cuando el FRANXX cambia de estado, ejecuta:

```
notificar()
```

y cada observador recibe la información correspondiente.

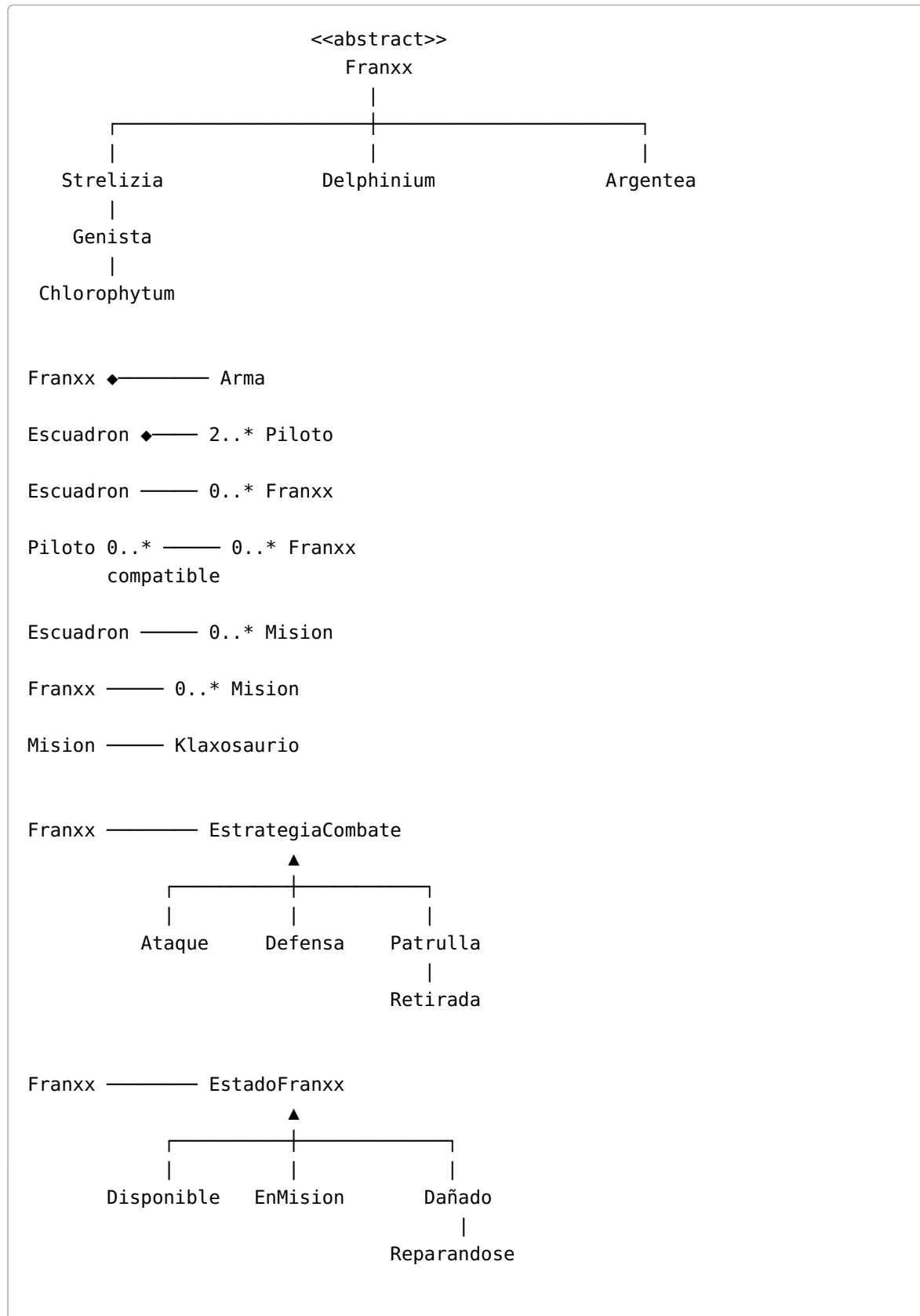
Justificación

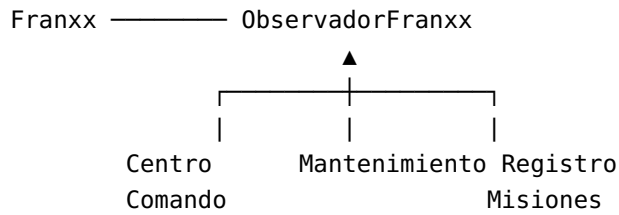
Observer evita que `Franxx` tenga que conocer directamente la implementación de cada sistema interesado.

Esto genera un diseño más flexible y con menor acoplamiento.

15. DIAGRAMA DE CLASES GENERAL

La estructura general del sistema puede representarse conceptualmente de la siguiente manera:





16. DIAGRAMA DE SECUENCIA — ATAQUE A UN KLAXOSAURIO

El caso de uso seleccionado es:

"Un escuadrón recibe una orden para atacar a un Klaxosaurio."

Los participantes son:

CentroComando
Escuadron
Piloto
Franxx
EstadoFranxx
EstrategiaCombate
Klaxosaurio

El flujo es el siguiente:

```

sequenceDiagram
    participant CentroComando
    participant Escuadron
    participant Franxx
    participant EstadoFranxx
    participant EstadoEnMision

    CentroComando->>Escuadron: ordenarAtaque()
    Escuadron->>Franxx: iniciarMision()
    Franxx->>EstadoFranxx: verificarEstado()
    EstadoFranxx->>EstadoEnMision: iniciarMision()
  
```

```

    | ejecutarEstrategia()
    ↓
EstrategiaAtaque
    |
    | atacar()
    ↓
Franxx
    |
    | atacar()
    ↓
Klaxosaurio
    |
    | recibirAtaque()
    ↓
Klaxosaurio
    |
    | resultado
    ↓
Franxx
    |
    | notificar()
    ↓
CentroComando
Mantenimiento
RegistroMisiones

```

Si el FRANXX se encuentra dañado, el estado correspondiente impide que se inicie la misión.

Por lo tanto, el comportamiento depende del patrón State.

17. ENCAPSULAMIENTO

Los atributos de las clases deben mantenerse privados.

Por ejemplo:

```

private nivelEnergia : double
private estado : EstadoFranxx
private pilotos : List<Piloto>

```

El acceso debe realizarse mediante métodos públicos cuando sea necesario.

Esto evita que otras clases modifiquen directamente el estado interno de los objetos.

Por ejemplo, no se debería permitir:

```
franxx.nivelEnergia = -500
```

sino utilizar un método que valide el valor:

```
franxx.reducirEnergia(cantidad)
```

De esta manera se mantiene la integridad de los objetos.

18. ABSTRACCIÓN

La clase `Franxx` es abstracta porque representa las características comunes de todos los modelos de FRANXX.

No es necesario conocer todos los detalles particulares para trabajar con un objeto FRANXX.

Por ejemplo:

```
Franxx franxx
```

puede representar:

```
Strelizia  
Delphinium  
Argentea  
Genista  
Chlorophytum
```

La abstracción permite trabajar con una interfaz común.

19. HERENCIA

La herencia permite reutilizar atributos y métodos comunes.

Por ejemplo:

```
Franxx  
  ↑  
Strelizia
```

`Strelizia` hereda las características comunes de `Franxx`, pero puede implementar comportamientos particulares.

Esto evita duplicar código.

20. POLIMORFISMO

El polimorfismo permite trabajar con diferentes tipos de `FRANXX` mediante una referencia común.

Por ejemplo:

```
Franxx f1 = new Strelizia();  
Franxx f2 = new Delphinium();  
Franxx f3 = new Argentea();
```

Los tres objetos pueden utilizar:

```
f1.ejecutarMision()  
f2.ejecutarMision()  
f3.ejecutarMision()
```

aunque internamente cada modelo pueda tener un comportamiento diferente.

Esto permite que el código cliente dependa de la abstracción `Franxx` y no de las clases concretas.

21. PRINCIPIO SRP — SINGLE RESPONSIBILITY PRINCIPLE

El principio de responsabilidad única establece que una clase debe tener una única responsabilidad principal.

En el diseño propuesto:

- `Franxx` administra el comportamiento general del robot.
- `Piloto` administra la información y comportamiento del piloto.
- `Mision` administra la información de una misión.
- `FranxxFactory` se ocupa de crear `FRANXX`.
- `EstadoFranxx` administra el comportamiento relacionado con el estado.
- `EstrategiaCombate` administra la estrategia de combate.

Por lo tanto, las responsabilidades se encuentran separadas.

22. PRINCIPIO OCP — OPEN/CLOSED PRINCIPLE

El sistema debe estar abierto para extensión pero cerrado para modificación.

Por ejemplo, si se desea agregar un nuevo FRANXX:

```
Raider
```

se puede crear:

```
class Raider extends Franxx
```

sin modificar necesariamente todos los FRANXX existentes.

De igual manera, para agregar una estrategia:

```
EstrategiaAtaqueEspecial
```

se puede crear una nueva clase que implemente:

```
EstrategiaCombate
```

Esto demuestra la aplicación del principio Open/Closed.

23. PRINCIPIO DIP — DEPENDENCY INVERSION PRINCIPLE

Las clases principales deberían depender de abstracciones y no de implementaciones concretas.

Por ejemplo:

```
Franxx → EstrategiaCombate
```

y no:

```
Franxx → EstrategiaAtaque
```

Esto permite reemplazar la estrategia sin modificar `Franxx`.

Lo mismo ocurre con:

```
Franxx → EstadoFranxx
```

en lugar de depender directamente de:

```
EstadoDisponible  
EstadoEnMision  
EstadoDañado
```

24. INCORPORACIÓN DEL FRANXX RAIDER

Si APE desea incorporar un nuevo modelo denominado `Raider`, se puede crear:

```
Raider extends Franxx
```

También puede incorporar una nueva estrategia:

```
EstrategiaRaider  
implements EstrategiaCombate
```

y una nueva arma:

```
ArmaEspecialRaider  
extends Arma
```

El patrón Factory permitirá crear el nuevo modelo.

Por ejemplo:

```
FranxxFactory.crearFranxx("RAIDER")
```

De esta forma, el sistema puede extenderse sin modificar las clases principales.

25. VENTAJAS DEL DISEÑO

El diseño propuesto presenta las siguientes ventajas:

Bajo acoplamiento

Las clases dependen principalmente de abstracciones.

Alta cohesión

Cada clase posee una responsabilidad específica.

Extensibilidad

Es posible agregar nuevos FRANXX, estados y estrategias.

Reutilización

La herencia permite reutilizar comportamiento común.

Polimorfismo

Los diferentes FRANXX pueden ser tratados mediante la clase abstracta `Franxx`.

Mantenibilidad

Los patrones de diseño separan responsabilidades y evitan clases excesivamente complejas.

Flexibilidad

Las estrategias y estados pueden cambiar durante la ejecución.

26. JUSTIFICACIÓN DE LOS PATRONES

Factory

Se utiliza para centralizar la creación de los diferentes modelos de FRANXX.

Problema: el código cliente no debería conocer cómo crear cada FRANXX.

Solución: delegar la creación a `FranxxFactory`.

Strategy

Se utiliza para encapsular las diferentes estrategias de combate.

Problema: existen diferentes comportamientos de combate.

Solución: cada comportamiento se representa mediante una estrategia independiente.

State

Se utiliza para representar los diferentes estados operativos del FRANXX.

Problema: el comportamiento cambia dependiendo del estado.

Solución: cada estado encapsula su propio comportamiento.

Observer

Se utiliza para notificar a diferentes sistemas cuando cambia el estado de un FRANXX.

Problema: varios objetos necesitan conocer los cambios.

Solución: el FRANXX notifica automáticamente a sus observadores.

27. RESPUESTA A LA PREGUNTA DEL SWITCH

Si se utilizara un `switch` dentro de `Franxx`, la clase tendría que conocer todas las estrategias posibles.

Por ejemplo:

```
switch(estrategia) {  
    case ATAQUE:  
        ...  
    case DEFENSA:  
        ...  
    case PATRULLA:  
        ...  
    case RETIRADA:  
        ...  
}
```

Esto genera varios problemas.

En primer lugar, la clase `Franxx` tendría demasiadas responsabilidades.

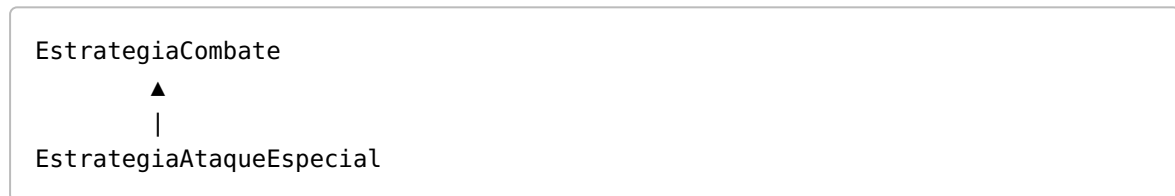
En segundo lugar, cada vez que se agregara una nueva estrategia habría que modificar la clase existente.

Esto dificulta el mantenimiento y viola principalmente el principio **Open/Closed**.

Con Strategy, cada estrategia se encuentra encapsulada en una clase independiente.

Por lo tanto, para agregar una nueva estrategia solamente se crea una nueva implementación de `EstrategiaCombate`.

Por ejemplo:



Sin necesidad de modificar la lógica principal de `Franxx`.

28. CONCLUSIÓN

El sistema desarrollado permite representar de manera orientada a objetos la gestión de FRANXX, pilotos, escuadrones y misiones.

La utilización de UML permite visualizar las clases, relaciones, responsabilidades y comportamientos del sistema antes de realizar su implementación.

Los patrones Factory, Strategy, State y Observer permiten solucionar diferentes problemas de diseño:

- Factory se encarga de la creación de FRANXX.
- Strategy permite cambiar las estrategias de combate.
- State controla los diferentes estados de un FRANXX.
- Observer permite notificar cambios a los diferentes sistemas interesados.

Además, mediante herencia, abstracción, encapsulamiento y polimorfismo se obtiene un diseño orientado a objetos flexible y reutilizable.

Finalmente, la aplicación de principios SOLID, especialmente SRP y OCP, permite que el sistema pueda crecer incorporando nuevos modelos de FRANXX, estrategias y comportamientos sin tener que modificar constantemente las clases existentes.

Por lo tanto, el diseño propuesto resulta adecuado para un sistema que necesita ser mantenible, extensible y adaptable a nuevos requerimientos.